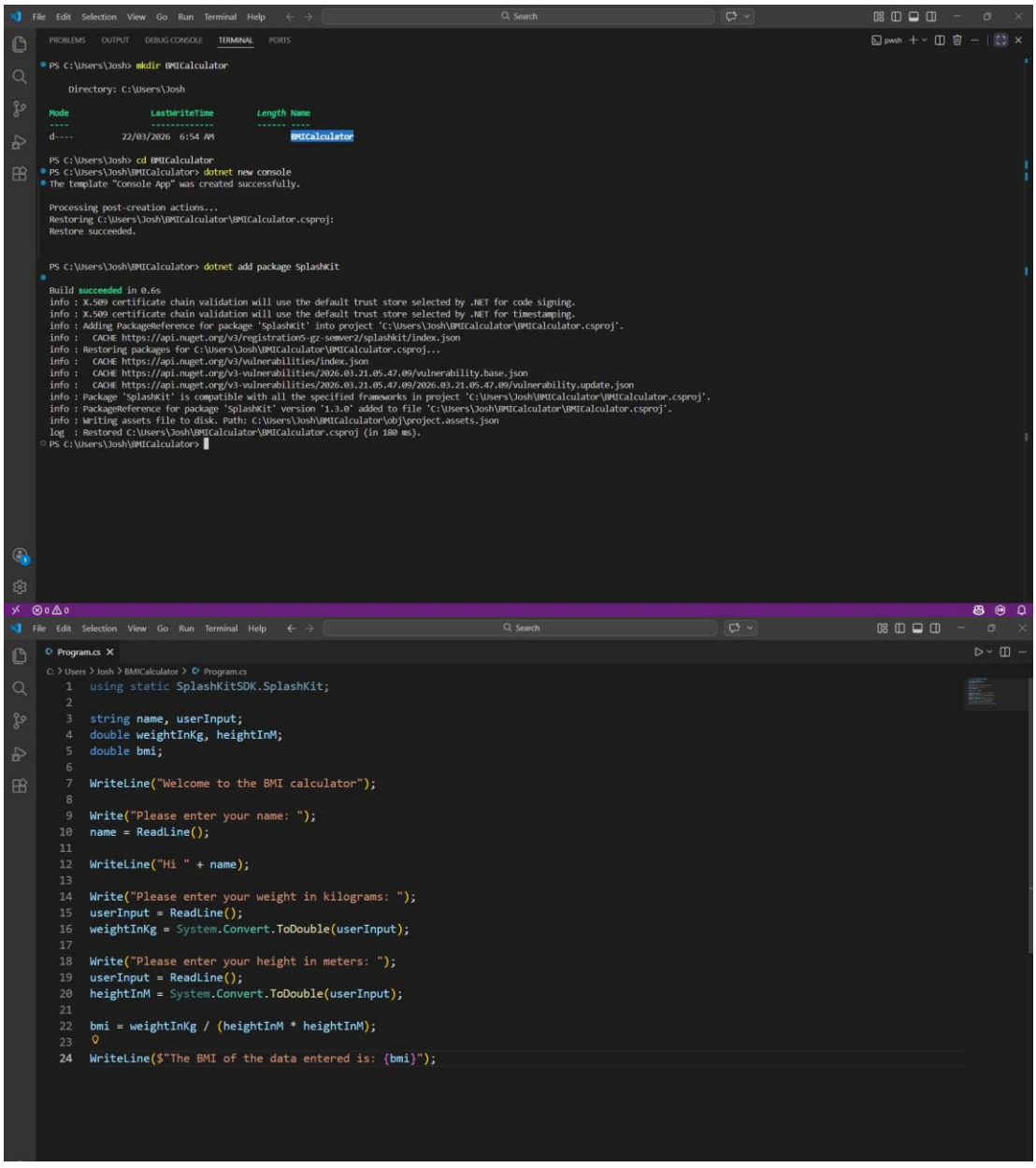
Sequence and Data

Name: Joshua Pijaca
Student ID: 223422713
Date: 22/03/2026

Learning Journey and Evidence

To learn about sequence and data I did the following.

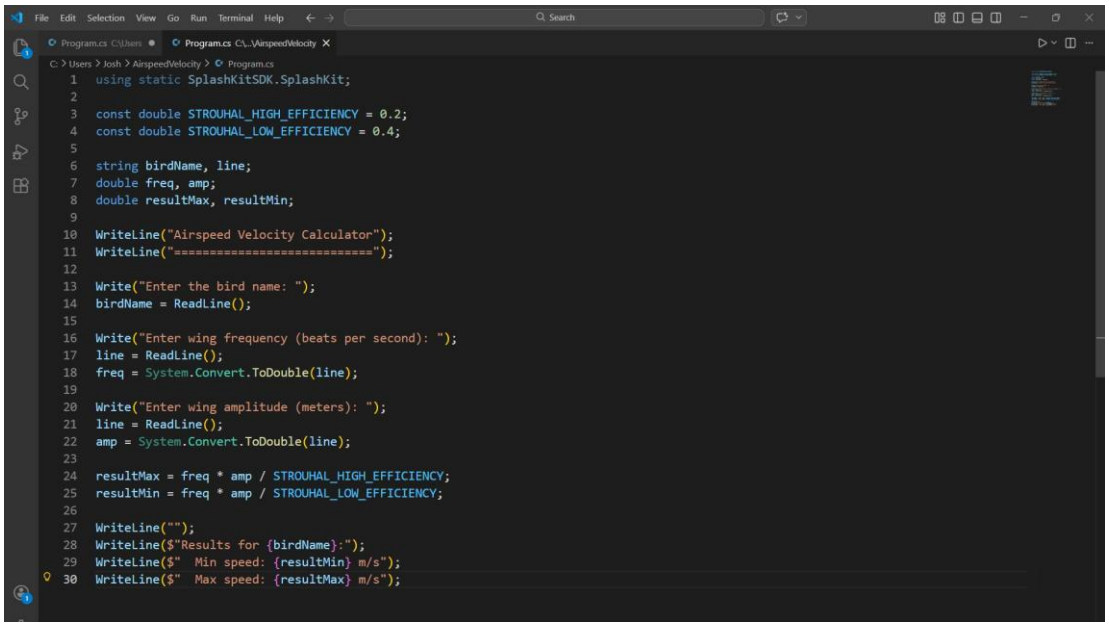
Created a BMI Calculator program that reads the user's name, weight, and height, then calculates and displays their BMI



```
PS C:\Users\Josh\BMICalculator> dotnet run
Welcome to the BMI calculator
Please enter your name: josh
Hi josh
Please enter your weight in kilograms: 90
Please enter your height in meters: 185
The BMI of the data entered is: 0.002629656683710738
PS C:\Users\Josh\BMICalculator>
```

This program helped me understand how sequence works in programming. The instructions run from top to bottom: first it displays a welcome message, then it prompts for input, reads the values, converts them from strings to numbers, performs a calculation, and outputs the result. I used variables like name, userInput, weightInKg, heightInM, and bmi to store different pieces of data. I also learned about assignment statements, where the result of a method call like ReadLine() is stored into a variable using the equals sign. One challenge I faced was that the ToDouble method existed in both SplashKit and System.Convert, causing an ambiguity error. I fixed this by using System.Convert.ToDouble() directly instead of importing both libraries.

Created an Airspeed Velocity Calculator that uses constants, variables, and mathematical expressions to calculate bird flight speed



```
using static SplashKitSDK.SplashKit;

const double STROUHAL_HIGH_EFFICIENCY = 0.2;
const double STROUHAL_LOW_EFFICIENCY = 0.4;

string birdName, line;
double freq, amp;
double resultMax, resultMin;

WriteLine("Airspeed Velocity Calculator");
WriteLine("============================");

Write("Enter the bird name: ");
birdName = ReadLine();

Write("Enter wing frequency (beats per second): ");
line = ReadLine();
freq = System.Convert.ToDouble(line);

Write("Enter wing amplitude (meters): ");
line = ReadLine();
amp = System.Convert.ToDouble(line);

resultMax = freq * amp / STROUHAL_HIGH_EFFICIENCY;
resultMin = freq * amp / STROUHAL_LOW_EFFICIENCY;

WriteLine("");
WriteLine($"Results for {birdName}:");
WriteLine($"  Min speed: {resultMin} m/s");
WriteLine($"  Max speed: {resultMax} m/s");
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Build succeeded in 0.6s
info : X.509 certificate chain validation will use the default trust store selected by .NET for code signing. …
PS C:\Users\Josh\AirspeedVelocity> dotnet run
Airspeed Velocity Calculator
==========================
Enter the bird name: European Swallow
Enter wing frequency (beats per second): 14
Enter wing amplitude (meters): 0.22

Results for European Swallow:
  Min speed: 7.7 m/s
  Max speed: 15.4 m/s
PS C:\Users\Josh\AirspeedVelocity>
```

This second program helped me understand constants and how they differ from variables. I declared STROUHAL_HIGH_EFFICIENCY and STROUHAL_LOW_EFFICIENCY as constants using the const keyword, because these values should never change while the program runs. The program also demonstrated more complex expressions, like freq * amp / STROUHAL_HIGH_EFFICIENCY, where multiple variables and constants are combined in a single calculation. I used string interpolation with the $ symbol to insert variable values directly into output strings, which was easier to read than using the + operator to concatenate strings.

Explored the Sequence and Data chapter concepts including programs, methods, variables, assignment statements, and expressions

```
Program.cs
C: > Users > Josh > BMICalculator > Program.cs
 1  using static SplashKitSDK.SplashKit;
 2  using static System.Convert;
 3
 4  string name, userInput;
 5  double weightInKg, heightInM;
 6  double bmi;
 7
 8  WriteLine("Welcome to the BMI calculator");
 9
10  Write("Please enter your name: ");
11  name = ReadLine();
12
13  WriteLine("Hi " + name);
14
15  Write("Please enter your weight in kilograms: ");
16  userInput = ReadLine();
17  weightInKg = ToDouble(userInput);
18
19  Write("Please enter your height in meters: ");
20  userInput = ReadLine();
21  heightInM = ToDouble(userInput);
22
23  bmi = weightInKg / (heightInM * heightInM);
24
25  WriteLine($"The BMI of the data entered is: {bmi}");
```

Reading through the Programmers Guide chapters helped me build a mental model of how programs work. A program is a sequence of instructions, and the computer executes them one at a time from top to bottom. Methods like WriteLine and ReadLine are reusable blocks of code from libraries that perform specific tasks. I learned that when you call a method, you pass it arguments in parentheses, and some methods return values you can store in variables. Variables need a type (like

string, int, or double) that determines what kind of data they hold. Assignment statements use the equals sign to store values, and expressions are anywhere in the code where a value is needed, such as in method arguments or on the right side of an assignment.

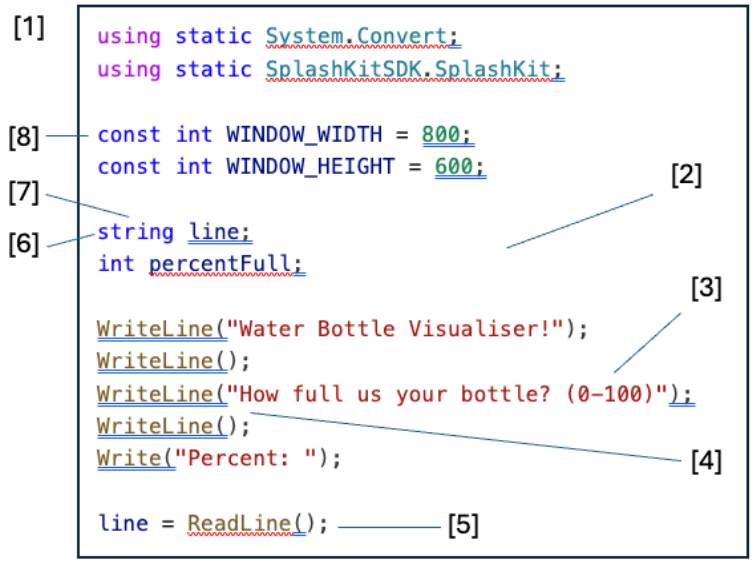
Brief Summary of Concepts

Concept	Key Idea / Concept
Program	A set of instructions stored in a file that the computer can run. Inside, a program is a sequence of instructions that get the computer to perform actions.
Variable	A named container that stores a value of a certain type. Its value can be changed as the program runs using assignment statements. For example, string name; creates a variable called name that stores text data.
Method	A reusable group of instructions that perform a specific task. Methods have a name, may accept arguments, and may return a value. For example, ReadLine is a method that reads text input from the user.
Method Call	A statement in your code that tells the computer to run a particular method. You write the method name followed by parentheses containing any arguments. For example, WriteLine("Hello") calls the WriteLine method with one string argument.
Assignment Statement	An instruction that stores a value in a variable. It uses the equals sign, with the variable on the left and the value on the right. For example, name = ReadLine(); stores the result of ReadLine in the name variable.
Expression	A piece of code that produces a value. Expressions can be literals (like "Hello" or 42), variable names, method calls that return values, or calculations using operators like + and *. They appear as arguments in method calls and as the value in assignment statements.

Consider the following terms and diagram. Then, fill in the table below.

1	This is code for a program.
2	Instructions in the code follow a sequence, starting from the first line.
3	This text is an argument, which is passed to the call to WriteLine.
4	This line of code contains a method call to the method named WriteLine.
5	This line of code contains an assignment statement. It calls the method named ReadLine and stores the result in the variable named line.
6	The code creates a variable named percentFull. It stores data with a type of "string".

7	This is the type. It determines the kind of data stored in the variable named line.
8	This is a constant named WINDOW_WIDTH. It stores an int value.



Expression	Method	Variable	Constant
Sequence	Type	Method Call	Argument
Program	Assignment Statement		

Reflection

What is the most important thing you learnt from this and why?

The most important thing I learned is how variables, assignment statements, and method calls work together to form a program. Variables act as named containers for data, assignment statements store values in those containers, and method calls let you use pre-built functionality from libraries. Understanding how these three concepts interact is essential because every program I write will use them. For example, in the BMI calculator, I used ReadLine to get user input, stored it in a variable with an assignment statement, converted it to a number using another method call, and then used an expression to calculate the result. Seeing how each concept connects to the others gave me a much clearer picture of how programs actually work.

What strategies did you use to help learn this material? How well did they work?

I learned by reading through the Programmers Guide chapters and then immediately building the sample programs myself. Rather than just reading the code examples, I typed them out by hand in VS Code and ran them to see the output. This hands-on approach worked well because it forced me to pay attention to every detail, like semicolons, types, and method names. When I encountered errors, such as the ambiguous ToDouble method call, debugging them taught me more than if the code had worked perfectly the first time. I also built a second program (the Airspeed Velocity Calculator) to reinforce the concepts in a different context, which helped solidify my understanding.

Based on what you have seen so far, what do you think your biggest challenge in this unit will be? Why? What strategies are you considering employing to overcome this?

I think my biggest challenge will be moving from following structured examples to designing my own programs from scratch. Right now I can follow along with guided examples and understand what each line does, but coming up with the sequence of steps to solve a new problem on my own will require more practice. I plan to tackle this by breaking problems down into small steps before writing any code, similar to how the Programmers Guide breaks the Airspeed Velocity program into an algorithm, building blocks, and pseudocode before actual coding. I also plan to keep attending seminars and using the SIT102 Teams channel to ask questions when I get stuck.